Component Telemetry — Schema & Integration Guide

Version: 1.1 (Review Draft)

Status: For Frontend + Backend review before implementation

Last updated: 2026-07-21 (v1.1 — added flush-reset semantics §6.6 and status/severity roll-up §6.7; §13 decisions D9–D10)

Backend repo: `oc-telemetry`

Frontend repo: `OC-Customer-UI-Portal` (branch `compliancePortal`)

Table of Contents

---

1. Overview

Goal

Capture a **component-level performance and activity log** for every tracked React component, focused on:

- **Backend API latency** — every API call made while the component is mounted (`latency_ms` , `http_code` , aggregated `api_cost`)
- **Frontend rendering cost** — React render/commit performance (`render_cost` : commits, total/max render ms, slow renders)
- **Component dwell time** — how long the component was mounted (`dwell_ms`)
- **Interaction signals** — clicks and form field activity (counts only, for context)

All correlated to a user session and business context (portal, entity, fund, account, identity).

> **Out of scope (v1):** field-change / PUT audit logging (`changed_fields`) — not included in this schema.

Recommended Approach: Hybrid

What	How	When sent
Component activity bundle	One `COMPONENT_SESSION` API call	On unmount, `pagehide` , or timer flush
Page navigation + load time	`PAGE_VIEW`	Immediate
JS crashes / errors	`GLOBAL_ERROR`	Immediate (never buffer)
Failed API calls (optional)	Standalone `API_REQUEST`	Immediate — for fast error alerting

Why not one call for everything?

- **Errors** must fire immediately — a crash may prevent unmount flush
- **Page views** are route-level, not owned by a single component
- **Failed APIs (optional)** may be sent immediately so ops sees 5xx without waiting for unmount
- **Existing latency reports** query individual `API_REQUEST` rows in BigQuery

The frontend sends **one call per component visit**; the backend optionally **fans out** nested events into separate BigQuery rows so existing queries keep working.

Design Principles

Rule	Detail
One component = one summary call	Sent on unmount, `pagehide`, or 60s timer for long-lived components
Never buffer	`GLOBAL_ERROR`, `PAGE_VIEW`; optional immediate send for failed APIs
Correlate everything	`session_id` + `component.instance_id` + `context`
No PII in interactions	Field **names** only — never capture input values
No BigQuery migration (v1)	All new fields live in `actor.additional_info` JSON column
Telemetry never breaks the app	Fire-and-forget, try/catch everywhere, no-op when URL unset

2. Architecture

```
                  FRONTEND (OC-Customer-UI-Portal)
┌──────────────────────────────────────────────────────────────────┐
│  While component is mounted:                                     │
│   ComponentSessionBuffer (in-memory)                             │
│      ├─ push API_REQUEST events                                  │
│      ├─ push UI_CLICK events                                     │
│      ├─ push FORM_INTERACTION events                             │
│      └─ accumulate render_cost / api_cost / interaction_cost     │
│                                                                  │
│  On flush (unmount | pagehide | timer):                          │
│      └─ POST /logs/api/ingest/component-session  (ONE call)      │
│                                                                  │
│  Immediate (never buffered):                                     │
│      ├─ POST /logs/api/ingest → PAGE_VIEW                        │
│      ├─ POST /logs/api/ingest → GLOBAL_ERROR                     │
│      └─ POST /logs/api/ingest → API_REQUEST (failures only, opt.)│
└──────────────────────────────────────────────────────────────────┘

                      BACKEND (oc-telemetry)
┌──────────────────────────────────────────────────────────────────┐
│  POST /logs/api/ingest/component-session                         │
│      ├─ Validate payload (Joi)                                   │
│      ├─ Insert 1 COMPONENT_SESSION summary row into BigQuery     │
│      └─ (Optional) Fan out events[] into individual BQ rows      │
└──────────────────────────────────────────────────────────────────┘
```

Flush Triggers

Trigger	`flush_reason`	Method	Buffer after send (§6.6)
Component unmounts	`"unmount"`	`fetch` or queue	final — buffer discarded
Tab close / navigation away	`"pagehide"`	`fetch(keepalive)` / `sendBeacon`	final — buffer discarded
Long-lived component (> 60s)	`"timer"`	`fetch`	**reset** — new segment starts
Debug / forced	`"manual"`	`fetch`	**reset** — new segment starts

3. API Endpoints

Endpoint	Purpose	When used
`POST /logs/api/ingest`	Single event (existing)	Errors, page views, optional failed API alerts
`POST /logs/api/ingest/batch`	Up to 50 events	Multiple page-level events
`POST /logs/api/ingest/component-session`	**New** — component bundle	Primary component logging

Request

```
POST /logs/api/ingest/component-session
Content-Type: application/json
Origin: <whitelisted portal URL>
```

Success Response

```
{
  "status": true,
  "error": null,
  "data": {
    "response": "logged",
    "log_id": "550e8400-e29b-41d4-a716-446655440000",
    "expanded_row_count": 6
  }
}
```

Field	Description
`log_id`	ID of the summary row inserted
`expanded_row_count`	Number of fan-out rows created (0 if fan-out disabled)

Error Response

```
{
  "status": false,
  "error": "Validation failed: actor.additional_info.component.instance_id is required",
  "data": null
}
```

HTTP status codes: `400` validation error, `413` payload too large, `500` server error.

4. Shared Envelope

Every event uses this top-level structure.

```
{
  "log_id": "550e8400-e29b-41d4-a716-446655440000",  // optional; server generates if missing
  "event_name": "COMPONENT_SESSION_WalletForm",
  "service": "oc_portal",
  "region": "asia-southeast1",
  "source_identifier": "gcp_prod_oc_portal",
  "status": true,

  "browser": {
    "url": "https://portal.one-constellation.com/entity-123/dashboard",
    "browser_time": "2026-07-20T12:00:00.000Z",
    "useragent": "Mozilla/5.0 ...",
    "userip": null,
    "user_location": null
  },

  "action": {
    "type": "COMPONENT_SESSION",
    "details": { /* see per-type sections */ }
  },

  "actor": {
    "type": "user",
    "user": { "id": "12345678-1234-1234-1234-123456789012" },
    "entity": { "id": "87654321-4321-4321-4321-210987654321" },
    "additional_info": { /* main extension point — see §5–§7 */ }
  },

  "file_details": null,
  "api_response": null,
  "descriptive_info": null,
  "log_severity": "INFO",

  "time_in_msec": 1753010400000,
  "time_out_msec": 1753010448210,
  "time_api_delay": 48210,
  "timestamp": 1753010448210000
}
```

Top-Level Field Reference

Field	Type	Required	Notes
`log_id`	string (uuid)	No	Server generates if absent. For `COMPONENT_SESSION` the server MUST assign this before fan-out (§9).
`event_name`	string	Yes	See §10 naming rules
`service`	string	Yes	e.g. `oc_portal`
`region`	string	No	Default: `asia-southeast1`
`source_identifier`	string	Yes	e.g. `gcp_prod_oc_portal`
`status`	boolean	Yes	`true` = success; `false` for errors. For `COMPONENT_SESSION`, derived server-side per §6.7.
`browser`	object	Yes	`userip` overwritten by server
`action.type`	enum	Yes	See §6–§8
`action.details`	object	Yes	Shape depends on `action.type`
`actor.type`	string	Yes	Usually `"user"`
`actor.user.id`	string (uuid)	Yes	`login_user_id` from localStorage
`actor.entity.id`	string (uuid)	No	From localStorage / context
`actor.additional_info`	object	Yes	All new telemetry fields
`log_severity`	enum	No	`INFO` | `WARN` | `ERROR` | `FATAL`. For `COMPONENT_SESSION`, derived server-side per §6.7.
`time_in_msec`	number	Yes	Start timestamp (epoch ms)
`time_out_msec`	number	Yes	End timestamp (epoch ms)
`time_api_delay`	number	No	Meaning varies by event type (see below)
`timestamp`	number	No	Epoch **microseconds** (= `time_out_msec` × 1000). Server normalizes for BigQuery.

Sentinel Values (Non-API Events)

For `COMPONENT_SESSION`, `PAGE_VIEW`, `GLOBAL_ERROR` — server accepts these sentinels in `action.details`:

```
{
  "api_tag": "<same as event_name>",
  "method": "NONE",
  "url": "component://WalletForm",
  "params": null,
  "body": null,
  "headers": null,
  "http_code": 0,
  "response_identifier": null
}
```

Event type	`url` sentinel	`time_api_delay` meaning
`COMPONENT_SESSION`	`component://{ComponentName}`	`dwell_ms`
`PAGE_VIEW`	`page://{route_pattern}`	`load_ms`
`GLOBAL_ERROR`	`error://{route_pattern}`	`0`
`API_REQUEST`	Real URL	API latency in ms

> **Note:** because `time_api_delay` is overloaded across event types, expose it through a typed BigQuery view that branches on `action.type` so analysts never misread it.

5. Common `additional_info` Block

Present on **every** telemetry event.

```
{
  "session_id": "b6b8f2e1-4c3d-4a5b-9e8f-1a2b3c4d5e6f",
  "context": {
    "portal_type": "customer",
    "entity_id": "87654321-4321-4321-4321-210987654321",
    "fund_id": "f-45",
    "account_id": "a-789",
    "identity_id": null,
    "route_pattern": "/:entityId/dashboard"
  },
  "telemetry_version": "1.1"
}
```

`context` Fields

Field	Type	Required	How to resolve
`portal_type`	`"customer"` | `"compliance"` | `"management"`	Yes	`getPortalType()`
`entity_id`	string | null	No	localStorage / URL `:entityId`
`fund_id`	string | null	No	localStorage (customer) or sessionStorage (compliance)
`account_id`	string | null	No	URL `:accountId` or account store
`identity_id`	string | null	No	URL `:identityId` or account store
`route_pattern`	string | null	No	Route pattern, **not** raw path with IDs

Resolution order:

1. Explicit override from call site (`telemetryMeta.context`)
2. URL pattern match via `matchPath()` against known routes
3. Store / localStorage / sessionStorage fallback
4. Unresolved → `null` (never guess, never throw)

`session_id`

- Generated once per browser tab: `crypto.randomUUID()`
- Stored in `sessionStorage` (survives reload, resets on new tab)
- Attached to every event. Note: this is a **per-tab** identifier, not a cross-tab user session.

6. Primary Schema: `COMPONENT_SESSION`

This is the **main payload** the frontend sends — one call per component visit (or per segment for long-lived components; see §6.6).

Complete Sample Payload

```
{
  "event_name": "COMPONENT_SESSION_WalletForm",
  "service": "oc_portal",
  "region": "asia-southeast1",
  "source_identifier": "gcp_eng_oc_portal",
  "status": true,

  "browser": {
    "url": "https://portal.one-constellation.com/87654321-.../account-setup/wallet",
    "browser_time": "2026-07-20T12:00:00.000Z",
    "useragent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36",
    "userip": null,
    "user_location": null
  },

  "action": {
    "type": "COMPONENT_SESSION",
    "details": {
      "api_tag": "COMPONENT_SESSION_WalletForm",
      "method": "NONE",
      "url": "component://WalletForm",
      "params": null,
      "body": null,
      "headers": null,
      "http_code": 0,
      "response_identifier": null
    }
  },

  "actor": {
    "type": "user",
    "user": { "id": "12345678-1234-1234-1234-123456789012" },
    "entity": { "id": "87654321-4321-4321-4321-210987654321" },
    "additional_info": {
      "session_id": "b6b8f2e1-4c3d-4a5b-9e8f-1a2b3c4d5e6f",
      "context": {
        "portal_type": "customer",
        "entity_id": "87654321-4321-4321-4321-210987654321",
        "fund_id": "f-45",
        "account_id": "a-789",
        "identity_id": null,
        "route_pattern": "/:entityId/account-setup/wallet"
      },
      "telemetry_version": "1.1",

      "component": {
        "instance_id": "c7a9e2b4-1f3d-4e5a-8b6c-9d0e1f2a3b4c",
        "name": "WalletForm",
        "file_path": "src/pages/AccountSetup/WalletForm.tsx",
        "parent_instance_id": "p1b2c3d4-5e6f-7890-abcd-ef1234567890",
        "lifecycle_phase": "unmount",
        "flush_reason": "unmount",
        "flush_seq": 1,
```

```
        "is_final": true
      },

      "summary": {
        "dwell_ms": 48210,
        "started_at_ms": 1753010400000,
        "ended_at_ms": 1753010448210,

        "render_cost": {
          "mount_to_first_paint_ms": 45,
          "commit_count": 12,
          "total_render_ms": 890,
          "max_render_ms": 120,
          "slow_render_count": 2,
          "profiler_enabled": true
        },

        "api_cost": {
          "call_count": 5,
          "success_count": 4,
          "error_count": 1,
          "total_latency_ms": 2340,
          "max_latency_ms": 890,
          "avg_latency_ms": 468,
          "api_tags": ["CAPI_GET_WALLET", "CAPI_UPDATE_WALLET"]
        },

        "interaction_cost": {
          "click_count": 8,
          "form_field_visits": 4,
          "validation_errors": 1,
          "unique_fields_visited": 3
        }
      },

      "events": [ /* see §7 — API_REQUEST, UI_CLICK, FORM_INTERACTION */ ],

      "truncated": {
        "events_dropped": 0,
        "events_included": 6,
        "events_max": 100
      }
    }
  },

  "time_in_msec": 1753010400000,
  "time_out_msec": 1753010448210,
  "time_api_delay": 48210,
  "log_severity": "INFO",
  "timestamp": 1753010448210000
}
```

`component` Object

Field	Type	Required	Description
`instance_id`	uuid	Yes	Unique per mount — generate with `crypto.randomUUID()`. Stable across all segments of one mount.
`name`	string	Yes	React component name, e.g. `WalletForm`
`file_path`	string	No	Source file path for debugging
`parent_instance_id`	uuid | null	No	Parent tracked component, if nested
`lifecycle_phase`	`"unmount"` | `"flush"`	Yes	`"unmount"` on final send, `"flush"` on interim segments
`flush_reason`	enum	Yes	`"unmount"` | `"pagehide"` | `"timer"` | `"manual"`
`flush_seq` (new v1.1)	number	Yes	1-based, monotonic per `instance_id`. Segment order for a multi-flush mount (§6.6).
`is_final` (new v1.1)	boolean	Yes	`true` on the last segment (unmount/pagehide), else `false` (§6.6).

`summary` Object

Section	Purpose
`dwell_ms`	Time component was mounted *during this segment*
`started_at_ms` / `ended_at_ms`	Absolute segment start/end timestamps
`render_cost`	Frontend rendering performance
`api_cost`	Aggregated API stats for this segment
`interaction_cost`	Clicks + form activity counts

`summary.render_cost`

Field	Type	Required	Description
`mount_to_first_paint_ms`	number	No	Time to first meaningful paint. Set on `flush_seq = 1` only; `null` after.
`commit_count`	number	Yes	React commit count while mounted
`total_render_ms`	number	Yes	Sum of Profiler `actualDuration`
`max_render_ms`	number	Yes	Slowest single render
`slow_render_count`	number	Yes	Renders exceeding 16ms (1 frame budget)
`profiler_enabled`	boolean	Yes	Whether React Profiler was active

`summary.api_cost`

Field	Type	Required	Description
`call_count`	number	Yes	Total API calls while mounted
`success_count`	number	Yes	HTTP 2xx responses
`error_count`	number	Yes	HTTP 4xx/5xx or network failure
`total_latency_ms`	number	Yes	Sum of all API latencies
`max_latency_ms`	number	Yes	Slowest single API call
`avg_latency_ms`	number	Yes	`total / call_count` (0 if no calls)
`api_tags`	string[]	No	Distinct API tags, max 20

`summary.interaction_cost`

Field	Type	Required	Description
`click_count`	number	Yes	Total clicks inside component
`form_field_visits`	number	Yes	Focus → blur pairs
`validation_errors`	number	Yes	Client-side validation error count
`unique_fields_visited`	number	No	Distinct form fields touched *within this segment* — see §6.6 (not summable across segments)

Buffer Limits

Limit	Value	Behavior when exceeded
Max nested events	100	Keep first 99 + last 1; set `truncated.events_dropped > 0`
Max `api_tags` in summary	20	Keep most frequent
Max click text length	100 chars	Truncate with `...`
Max payload size	1 MB	Backend returns `413`

```
"truncated": {
  "events_dropped": 12,
  "events_included": 100,
  "events_max": 100,
  "reason": "event_cap_exceeded"
}
```

> **Transport note:** the client-side 100-event cap must also keep the serialized payload under the transport limit for `navigator.sendBeacon` (~64KB in several browsers), not just the 1MB server cap. On `pagehide`, prefer `fetch(url, { keepalive: true })` with `sendBeacon` as fallback,

> since a JSON-content-type beacon can trigger a CORS preflight that may not complete during unload, and a `413` is invisible to `sendBeacon`.

6.6 Flush Semantics (Buffer Reset Model)

New in v1.1. A long-lived component flushes on the 60s timer and again on unmount/pagehide. Flushes are **delta-based ("snapshot-and-reset")**: each flush sends only what accumulated **since the last flush**, then resets. This bounds payload and memory (the whole point of the timer) and avoids re-sending events.

On any flush (`timer`, `manual`, `pagehide`, `unmount`):

1. Send the buffer accumulated since the last flush (or since mount, for the first).
2. Set `lifecycle_phase = "unmount"` when `flush_reason` is `unmount` / `pagehide`; otherwise `"flush"`.
3. If the mount is **not** ending: reset counters and `events[]`, keep the same `instance_id`, increment `flush_seq`, and set a fresh `started_at_ms = now`.

`events[].seq` stays **globally monotonic across the whole mount** (it does not reset per segment), so ordering is preserved end to end.

Per-field aggregation. Each row describes one *segment*. Consumers join on `instance_id` and aggregate a full visit as follows:

Field	Per-row meaning	Full-visit aggregation
`dwell_ms`	duration of this segment	`SUM(dwell_ms)`
`started_at_ms` / `ended_at_ms`	segment boundaries	`MIN(started_at_ms)`, `MAX(ended_at_ms)`
`commit_count`, `total_render_ms`, `slow_render_count`	delta this segment	`SUM(...)`
`max_render_ms`, `max_latency_ms`	max within segment	`MAX(...)`
`mount_to_first_paint_ms`	set on `flush_seq = 1` only	take the non-null value
`call_count`, `success_count`, `error_count`, `total_latency_ms`	delta this segment	`SUM(...)`
`avg_latency_ms`	segment average (informational)	recompute: `SUM(total_latency_ms) / NULLIF(SUM(call_count), 0)`
`interaction_cost.*` counts	delta this segment	`SUM(...)`
`unique_fields_visited`	distinct within segment	**not summable** — true distinct needs raw field events; treat as approximate
`events[]`	events in this segment	concatenate ordered by `seq`

> Two fields are deliberately **not summable** across segments: `unique_fields_visited` and any distinct count. Document this so analysts never `SUM` them by accident. To identify the closing row of a multi-segment mount, filter `is_final = true` rather than computing `MAX(flush_seq)`.

6.7 Status & Severity Roll-Up

New in v1.1. For a bundled `COMPONENT_SESSION`, top-level `status` and `log_severity` are **derived and server-authoritative** — the server recomputes them from `events[]` and overrides whatever the client sent (payloads are untrusted).

Severity ladder: `INFO < WARN < ERROR < FATAL`.

- **`log_severity`** = maximum severity across the segment's nested `events[]`, defaulting to `INFO` when there are none.
- **`status`** = `false` if `log_severity ≥ ERROR`, otherwise `true`.

Segment contains	`log_severity`	`status`	Rationale
only successful events	`INFO`	`true`	clean
a form `validation_error` (WARN)	`WARN`	`true`	user input mistake ≠ system failure
a 4xx/5xx API call (ERROR)	`ERROR`	`false`	technical failure
a crash surfaced in-bundle (FATAL)	`FATAL`	`false`	technical failure

Consistency invariant to assert in backend (Joi) validation:

```
api_cost.error_count > 0
 ⇔ at least one nested event with status = false
 ⇔ log_severity ≥ ERROR
 ⇔ status = false
```

Full-visit roll-up (with §6.6): visit severity = `MAX(log_severity)` across rows; visit had a failure = `LOGICAL_OR(NOT status)` / `MAX(error_count) > 0`.

7. Nested Events (`events[]`)

Each item in `events[]` shares these base fields:

Field	Type	Required	Description
`seq`	number	Yes	1-based order within session (monotonic across all segments)
`type`	enum	Yes	`API_REQUEST` | `UI_CLICK` | `FORM_INTERACTION`
`ts_ms`	number	Yes	When event occurred (epoch ms)
`status`	boolean	Yes	Success/failure of that action
`log_severity`	enum	Yes	`INFO` | `WARN` | `ERROR`
`event_name`	string	Yes	Normalized event name

7.1 Nested `API_REQUEST`


```
{
  "seq": 5,
  "type": "API_REQUEST",
  "ts_ms": 1753010415000,
  "status": true,
  "log_severity": "INFO",
  "event_name": "CAPI_UPDATE_WALLET",
  "api_tag": "CAPI_UPDATE_WALLET",
  "method": "PUT",
  "url": "https://api.one-constellation.com/wallet/update/abc",
  "http_code": 200,
  "latency_ms": 890,
  "time_in_msec": 1753010415000,
  "time_out_msec": 1753010415890,
  "response_identifier": "",
  "file_details": { "file_name": "WalletForm.tsx", "function_name": "handleSubmit()" },
  "api_response": { "http_status_code": 500, "http_error_response": "..." }
}
```

Field	Type	Required	Notes
`api_tag`	string	Yes	From existing `APIEventName` map
`method`	HTTP verb	Yes	GET, POST, PUT, DELETE, etc.
`url`	string (uri)	Yes	Full API URL
`http_code`	number	Yes	Response status code
`latency_ms`	number	Yes	Backend latency: `time_out_msec − time_in_msec`
`time_in_msec`	number	Yes	Request start
`time_out_msec`	number	Yes	Response received
`response_identifier`	string	No	Backend correlation ID
`file_details`	object	No	Call-site source location
`api_response`	object	No	Failures only (4xx/5xx)

> **v1 decision:** Request `body`, `params`, and `headers` are **excluded** from nested events to reduce PII risk.

7.2 Nested `UI_CLICK`

```
{
  "seq": 2,
  "type": "UI_CLICK",
  "ts_ms": 1753010405000,
  "status": true,
  "log_severity": "INFO",
  "event_name": "UI_CLICK_edit_wallet",
  "track_id": "edit_wallet",
  "element": {
    "tag": "BUTTON",
    "text": "Edit Wallet",
    "role": null,
    "data_track_id": "edit_wallet"
  }
}
```

Field	Type	Required	Notes
`track_id`	string	Yes	From `data-track-id` or derived (define the fallback derivation to avoid unstable/high-cardinality ids)
`element.tag`	string	Yes	HTML tag name
`element.text`	string	No	Truncated to 100 chars — apply redaction, text can carry PII
`element.role`	string	No	ARIA role if present
`element.data_track_id`	string	No	Explicit tracking attribute

Capture targets: `button`, `a`, `[role=button]`, `[data-track-id]`
Opt-out: `data-track="off"` on element or ancestor
Never capture: input values, passwords, clipboard content

7.3 Nested `FORM_INTERACTION`

```
{
  "seq": 3,
  "type": "FORM_INTERACTION",
  "ts_ms": 1753010406000,
  "status": true,
  "log_severity": "INFO",
  "event_name": "FORM_WalletForm_field_visit",
  "form": "WalletForm",
  "field": "walletAddress",
  "kind": "field_visit",
  "time_in_field_ms": 5230,
  "error": null
}
```

Field	Type	Required	Notes
`form`	string	Yes	Form/component name
`field`	string	Yes	Field name/id only — **never the value**
`kind`	enum	Yes	See below
`time_in_field_ms`	number	No	Required for `field_visit`
`error`	string	No	Error code for `validation_error` only

`kind` values: `field_visit` (focus → blur with duration), `validation_error` (client-side validation failed), `blur` (blur without prior focus tracking — fallback).
Capture targets: `input`, `select`, `textarea` (delegated `focusin` / `focusout`)

8. Immediate-Send Events

These use `POST /logs/api/ingest` and are **not** nested inside `COMPONENT_SESSION`.

8.1 `PAGE_VIEW`

Sent on every route change via `TelemetryRouteListener`. Adds these `context`-level fields inside `additional_info`:

Field	Type	Required
`route`	string	Yes — route pattern
`prev_route`	string	No
`nav_type`	`"PUSH"` | `"POP"` | `"REPLACE"`	Yes
`load_ms`	number	Yes

8.2 `GLOBAL_ERROR`

Sent immediately — never buffered.

```
{
  "event_name": "GLOBAL_ERROR_react_error_boundary",
  "status": false,
  "action": { "type": "GLOBAL_ERROR", "details": { "url": "error:///:entityId/dashboard", "http_code": 0
  "actor": {
    "additional_info": {
      "component": { "instance_id": "c7a9e2b4-...", "name": "WalletForm" },
      "error": {
        "kind": "react_error_boundary",
        "message": "Cannot read properties of undefined (reading 'id')",
        "stack": "TypeError: ...\n at WalletForm (WalletForm.tsx:214:18)",
        "component_stack": "in WalletForm\n in AccountSetup\n in CustomerLayoutShell",
        "source": "WalletForm.tsx",
        "line": 214,
        "col": 18
      }
    }
  },
  "log_severity": "FATAL"
}
```

Kind	Severity	Source
`uncaught_exception`	ERROR	`window.onerror`
`unhandled_rejection`	ERROR	`window.unhandledrejection`
`react_error_boundary`	FATAL	`<TelemetryErrorBoundary>`

> **PII note:** `error.message` and `error.stack` can carry user values; run a redaction pass before send. Likewise, `browser.url` still contains raw entity/account UUIDs even though event names use `route_pattern` — strip or patternize `browser.url` server-side, or accept it as identifying data.

8.3 Standalone `API_REQUEST` — Failed Calls (Optional, Immediate)

Successful API calls are captured inside `COMPONENT_SESSION.events[]`. Failed calls (4xx/5xx or network error) **may also** be sent immediately via `POST /logs/api/ingest` so alerting does not wait for component unmount. All successful API latency data stays in the component buffer — no separate immediate call needed per request.

9. Backend Fan-Out (Optional)

When the backend receives a `COMPONENT_SESSION` payload, it **may** expand nested `events[]` into individual BigQuery rows:

Source	BigQuery row `action.type`
Top-level payload	`COMPONENT_SESSION` (summary row)
Each `events[]` where `type = API_REQUEST`	`API_REQUEST`
Each `events[]` where `type = UI_CLICK`	`UI_CLICK`
Each `events[]` where `type = FORM_INTERACTION`	`FORM_INTERACTION`

All expanded rows inherit: `session_id`, `context`, `component.instance_id`, and `additional_info.parent_log_id` (reference to the summary row's server-assigned `log_id`). This keeps existing latency report queries working without requiring the frontend to send multiple calls.

10. Event Naming Conventions

Type	Pattern	Example
Component session	`COMPONENT_SESSION_{ComponentName}`	`COMPONENT_SESSION_WalletForm`
API	Existing `APIEventName` map	`CAPI_GET_WALLET`
Page view	`PAGE_VIEW_{route_slug}`	`PAGE_VIEW_entity_dashboard`
Click	`UI_CLICK_{track_id}`	`UI_CLICK_edit_wallet`
Form	`FORM_{FormName}_{kind}`	`FORM_WalletForm_field_visit`
Error	`GLOBAL_ERROR_{kind}`	`GLOBAL_ERROR_react_error_boundary`

Rules: use route **patterns**, not raw UUIDs, in event names; `track_id` = lowercase snake_case; max length 128 characters.

11. Environment Flags

Add to `envs/*.env` in the frontend repo:

```
# Master switch — unset = all telemetry off (existing)
VITE_LOGS_API_URL=https://telemetry.one-constellation.com/logs/api

# Component session bundle
VITE_TELEMETRY_COMPONENT_SESSION=true      # default: false

# Immediate events
VITE_TELEMETRY_PAGE_VIEWS=true             # default: false
VITE_TELEMETRY_ERRORS=true                 # default: false
VITE_TELEMETRY_FAILED_API_IMMEDIATE=true   # default: false — optional 5xx alert path

# Buffer tuning
VITE_TELEMETRY_COMPONENT_FLUSH_MS=60000    # timer flush for long-lived components
VITE_TELEMETRY_COMPONENT_EVENT_CAP=100     # max nested events per session
```

Rollout: enable flags per environment (`eng.env` → `staging.env` → production).

> **Consent note:** the master URL switch is not the same as per-user tracking consent. If any target jurisdiction requires consent (relevant for the `compliancePortal` branch), gate emission on consent as well.

12. TypeScript Interfaces

Copy into `networking/telemetry/types.ts` in the frontend repo.

```
type PortalType = 'customer' | 'compliance' | 'management';
type ActionType = 'COMPONENT_SESSION' | 'API_REQUEST' | 'PAGE_VIEW' | 'GLOBAL_ERROR';
type NestedEventType = 'API_REQUEST' | 'UI_CLICK' | 'FORM_INTERACTION';
type LogSeverity = 'INFO' | 'WARN' | 'ERROR' | 'FATAL';
type FlushReason = 'unmount' | 'pagehide' | 'timer' | 'manual';
type FormInteractionKind = 'field_visit' | 'validation_error' | 'blur';
type ErrorKind = 'uncaught_exception' | 'unhandled_rejection' | 'react_error_boundary';

interface TelemetryContext {
  portal_type: PortalType;
  entity_id: string | null;
  fund_id: string | null;
  account_id: string | null;
  identity_id: string | null;
  route_pattern: string | null;
}

interface ComponentRef {
  instance_id: string;
  name: string;
  file_path?: string;
  parent_instance_id?: string | null;
  lifecycle_phase?: 'unmount' | 'flush';
  flush_reason?: FlushReason;
  flush_seq: number;        // new v1.1 — 1-based, monotonic per instance_id
  is_final: boolean;        // new v1.1 — true on last segment
}

interface RenderCost {
  mount_to_first_paint_ms?: number;
  commit_count: number;
  total_render_ms: number;
  max_render_ms: number;
  slow_render_count: number;
  profiler_enabled: boolean;
}

interface ApiCost {
  call_count: number;
  success_count: number;
  error_count: number;
  total_latency_ms: number;
  max_latency_ms: number;
  avg_latency_ms: number;
  api_tags?: string[];
}

interface InteractionCost {
  click_count: number;
  form_field_visits: number;
  validation_errors: number;
  unique_fields_visited?: number;
}
```

```
interface ComponentSummary {
  dwell_ms: number;
  started_at_ms: number;
  ended_at_ms: number;
  render_cost: RenderCost;
  api_cost: ApiCost;
  interaction_cost: InteractionCost;
}

interface NestedEventBase {
  seq: number;
  type: NestedEventType;
  ts_ms: number;
  status: boolean;
  log_severity: LogSeverity;
  event_name: string;
}

interface NestedApiEvent extends NestedEventBase {
  type: 'API_REQUEST';
  api_tag: string;
  method: string;
  url: string;
  http_code: number;
  latency_ms: number;
  time_in_msec: number;
  time_out_msec: number;
  response_identifier?: string;
  file_details?: { file_name: string; function_name: string };
  api_response?: { http_status_code: number; http_error_response: string };
}

interface NestedClickEvent extends NestedEventBase {
  type: 'UI_CLICK';
  track_id: string;
  element: { tag: string; text?: string; role?: string | null; data_track_id?: string };
}

interface NestedFormEvent extends NestedEventBase {
  type: 'FORM_INTERACTION';
  form: string;
  field: string;
  kind: FormInteractionKind;
  time_in_field_ms?: number | null;
  error?: string | null;
}

type NestedEvent = NestedApiEvent | NestedClickEvent | NestedFormEvent;
```

13. Open Decisions for Review

Please confirm these before implementation begins.

#	Decision	Options	Recommendation
D1	Failed API (5xx) immediate send	Off / on	**On** in staging for faster error visibility
D2	Nested API body/params	Include / exclude	**Exclude** in v1 (PII risk)
D3	Backend fan-out	Yes / No	**Yes** — keeps latency queries working
D4	Timer flush interval	30s / 60s / 120s	**60s**
D5	Max nested events	50 / 100 / 200	**100**
D6	Long-lived layout components	Track / exclude	**Exclude** shell layouts
D7	Click/form interaction tracking	On / off	**On** — counts feed `interaction_cost` summary
D8	Click text capture	Full / truncated / omit	**Truncated to 100 chars**
D9	Timer-flush buffer model (§6.6)	Delta / cumulative	**Delta (snapshot-and-reset)** — bounded payload; add `flush_seq` + `is_final`
D10	Session `status` / `log_severity` derivation (§6.7)	Client / server-authoritative	**Server-authoritative** — derive from `events[]` ; assert consistency invariant

14. Implementation Checklist (Frontend)

Suggested file structure in `OC-Customer-UI-Portal` :

```
networking/telemetry/
├── types.ts                      # TypeScript interfaces (§12)
├── session.ts                    # getOrCreateSessionId()
├── context.ts                    # resolveTelemetryContext()
├── flags.ts                      # Env flag helpers
├── ComponentSessionBuffer.ts     # In-memory buffer per component instance
├── useComponentTelemetry.ts      # Hook: mount → track → unmount flush
├── ComponentTelemetryContext.tsx # React context: current component stack
├── sendComponentSession.ts       # POST /ingest/component-session
├── sendImmediateEvent.ts         # POST /ingest (errors, page views, optional failed APIs)
└── index.ts                      # Public exports
```

Step-by-step

1. Create `types.ts` with interfaces from §12
2. Implement `session.ts` — `sessionStorage`-backed `session_id`
3. Implement `context.ts` — 4-step resolver from §5
4. Implement `ComponentSessionBuffer.ts` — push events, compute summary, apply delta-reset on flush (§6.6)

5. Implement `useComponentTelemetry.ts` — mount/unmount lifecycle
6. Wire API wrappers to push `API_REQUEST` events into active buffer
7. Wire click/form delegated listeners to push into active buffer
8. Implement flush on unmount + `pagehide` (`fetch keepalive` / `sendBeacon`) + timer
9. Implement immediate sends for `GLOBAL_ERROR` and `PAGE_VIEW`
10. (Optional) Immediate send for failed API calls (`VITE_TELEMETRY_FAILED_API_IMMEDIATE`)
11. Add env flags to `envs/eng.env` ; test against staging backend
12. Pilot on 2–3 key components (e.g. `WalletForm` , `IdentitySelection`)
13. Confirm open decisions (§13, incl. D9–D10) with backend team

What NOT to track

- Shell/layout components that never unmount (`CustomerLayoutShell` , etc.)
- UI primitives (buttons, inputs from a shared library)
- Components marked with `/* @no-telemetry */` comment (Phase 8b)

15. Frontend Impact & Integration Risk

New in v1.1. This section grounds the plan in the current state of `OC-Customer-UI-Portal` (branch `compliancePortal`) so both teams can see the change footprint and the risk to existing flows before work starts.

Headline: this is an expansion, not a greenfield build

The repo already ships the core ingest pipeline this plan depends on. These are **reused as-is**:

Existing asset	Location	Reused for
`ingestEvent()` — fire-and-forget POST, no-ops when URL unset	`networking/IngestApi.ts`	Immediate sends + component-session send; same payload shape
URL→pattern normalizer (strips UUIDs/ prefixes/query)	`networking/EventNames/ utils.ts`	Route-pattern extraction for `PAGE_VIEW` / `route_pattern`
`getPortalType()` (hostname-based)	`src/lib/portalUtils.ts`	`context.portal_type`
`VITE_LOGS_API_URL` (set in eng/ staging)	`envs/*.env` , typed in `src/ custom.d.ts`	Master telemetry switch
localStorage identity keys	`login_user_id` , `entity_id` , `fund_uuid`	`actor.user.id` , `context.entity_id` , `context.fund_id`

Change footprint

Approximately **~12 new files** and **~15 existing files touched** for the core infra, then per-component opt-in.

Category	Files	Nature	Risk
New telemetry module	`networking/telemetry/*` (~10 files, §14)	Purely additive — nothing depends on them	Low
New components	`TelemetryRouteListener`, `TelemetryErrorBoundary`	Additive; error boundary built from scratch (none exists today)	Low
API instrumentation	The **7 axios wrappers**: `RegionApiClient`, `CentralNetwork`, `ComplianceNetwork`, `PortfolioNetwork`, `NotificationNetwork`, `MappingNetwork`, `ReportsApi`	Shared modification — push `API_REQUEST` events into active buffer	**High**
App wiring	`src/App.tsx`, layout shells (`CustomerLayoutShell`, `ComplianceLayout`)	Mount route listener + wrap error boundary	Medium
Config	`src/custom.d.ts` + 4× `envs/*.env`	Add `VITE_TELEMETRY_*` flags (none exist yet)	Low
Per-component adoption	Each tracked component (pilot: 2–3)	Add `useComponentTelemetry()` + Profiler; not structural	Low, incremental

Risk to existing flows (ranked)

#	Risk	Why it matters	Severity
1	API-wrapper instrumentation	All ~180 API operations + the 401→`performLogout` flow + `AbortController` cancellation run through the 7 wrappers. They are **not identical** (Compliance wrapper already lacks the abort-guard/logout latch), so the change is applied 7× to drifted code. A bug here degrades real API traffic, not telemetry.	High
2	**Component attribution of API calls**	Services are called from hooks, **not bound to a mounted component**, so "which component owns this API call" is genuinely ambiguous. This is the least-specified, riskiest part of the plan and needs a defined ownership rule (e.g. current-component-context stack) before coding.	High
3	Error-boundary behavior change	No boundary exists today — an uncaught render error white-screens. Adding one changes app-wide crash behavior (an improvement) but the fallback UI must be themed/i18n'd or you replace a white screen with a broken one.	Medium
4	Route-pattern fidelity	Using classic `<BrowserRouter><Routes>` (not the v7 data router) means **no built-in matched-pattern hook**. Must regex-normalize paths (reuse `EventNames/utils.ts`). Listener is read-only, so it can't break flows — only produce inconsistent pattern data.	Low–Med
5	React Profiler overhead	Production profiling build adds per-render cost; keep scoped to pilot components (see §1/§6.6).	Low

Containment requirements (mandatory)

Three existing properties keep the high-risk item's runtime blast radius near zero — treat them as hard requirements for the new code too:

- **Flag-gated, default-off:** every new path behind a `VITE_TELEMETRY_*` flag defaulting to `false`.
- **Fire-and-forget in try/catch:** the API-layer hook must never throw into app flow, exactly like `ingestEvent` today.
- **Safe no-op:** when the URL is unset *or* there is no active component buffer, do nothing — never block, retry, or error the underlying call.

Known gaps to resolve before coding

- **No test harness:** the repo has no unit-test framework (only bespoke verify scripts), so changes to the 7 shared wrappers need **manual QA per portal** (customer / compliance / management), especially login/logout and request cancellation.
- **Key-name mismatch:** the plan's `context.fund_id` maps to localStorage key **`fund_uuid`**, not `fund_id` — wire carefully.
- **`networking/` lives outside `src/`** and is reached by relative paths; the `@` alias only covers `src`. New telemetry code importing the ingest layer will use relative paths too.
- **Two portal-type enums exist** (`PortalType` vs `CompliancePortalType`) — use `getPortalType()` for telemetry tagging, not the banner enum.

Quick Reference

```
User opens page
   ↳▶ PAGE_VIEW                          (immediate → /ingest)

User enters WalletForm
   ├▶ Buffer starts (instance_id created)
   ├▶ API calls    → push to buffer (latency_ms per call)
   ├▶ Renders      → accumulate render_cost (Profiler)
   ↳▶ Clicks/forms → push to buffer (interaction counts)

Long-lived (> 60s)                       (timer flush → delta segment, buffer resets)

User leaves WalletForm
   ↳▶ COMPONENT_SESSION                   (one call → /ingest/component-session, is_final=true)
         ├─ summary (render_cost + api_cost + dwell_ms)
         └─ events[] (API + interaction timeline)

Failed API (optional)
   ↳▶ API_REQUEST                         (immediate → /ingest, errors only)

App crashes
   ↳▶ GLOBAL_ERROR                        (immediate → /ingest)
```

Contact & References

- **Backend repo:** `oc-telemetry`
- **Frontend repo:** `OC-Customer-UI-Portal` (branch `compliancePortal`)
- **Existing ingest endpoint:** `POST /logs/api/ingest`
- **New endpoint:** `POST /logs/api/ingest/component-session`
- **BigQuery dataset:** `telemetry_{PROJECT_ENV}`
- **BigQuery table:** `{service}-{region}-Partitioned`

For questions on backend validation or fan-out behavior, contact the backend team before finalizing the open decisions in §13.